

Sunbeam Institute of Information Technology
Pune and Karad

Algorithms and Data structures

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Data Structure

- organising data inside memory for efficient processing along with operations like add, delete, search, etc which can be performed on data.

e.g. stack - push/pop/peek

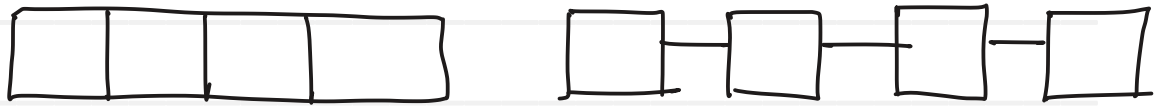
- data structures are used to achieve
  - Abstraction
  - Reusability
  - Efficiency

} Abstract Data Types (ADT)

(time & space)

Physical Data structures

- will tell about physical orgnization of data inside memory



Array Linked List

Logical Data structures

- will tell about the relationship bet" data
- always implemented with the help of physical data structures.

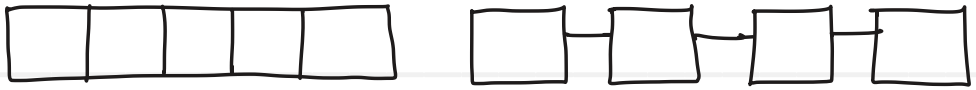
e.g. stack, queue, tree, heap, graph

Types of data structures

(Basic)

Linear data structures

- data is organised sequentially/ linearly
- data can be accessed sequentially



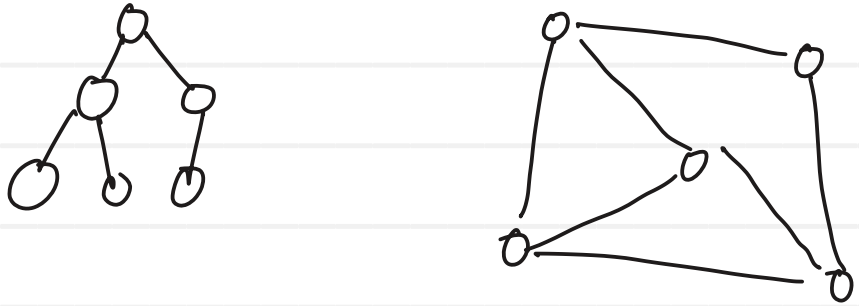
e.g. class / struct
Array
Linked List

Stack
Queue

(Advanced)

Non linear data structures

- data is organised in multiple levels (hierarchy)
- data can not be accessed sequentially



e.g. tree , heap , graph

Hash Table

Algorithm

- Algorithm is set of instructions given to the programmers
- Algorithm is step by step solution of given problem statement
- As written in human understandable languages, algorithms are programming languages independent
- Algorithms are used as templates and can be

Algorithm (Template) ⟶ Program (Implementation)

Problem - Find sum of array elements

Algorithm :

1. create space inside memory to store sum, initialize it to 0
2. Traverse array from one end to another
3. add current element into previously calculated sum
4. once array is finished, print or return final sum.

e.g. Searching :- Linear / binary
Sorting :- selection / bubble / insertion / merge / quick / heap

Algorithm analysis

- it is done for efficiency measurement and also known as time/space complexity

- It is done to finding time and space requirements of the algorithm
  a. Time - time required to execute the algorithm (us, ns, ms, s....)
  b. Space - space required to execute the algorithm inside memory (bytes, Kb, Mb...)

- finding exact time and space of the algorithm is not possible because it depends on few external factors like
  - time is dependent on type of machine (CPU), number of processes running at that time
  - space is dependent on type of machine (architecture), data types

Asymptotic analysis

- Approximate time and space analysis of the algorithm is always done
- Mathematical approach is used to find time and space requirements of the algorithm and it is known as "Asymptotic analysis" → growth rate of time & space
- Asymptotic analysis also tells about behaviour of the algorithm for different input or for change in sequence of input
- This behaviour of the algorithm is observed in three different cases
  a. Best case
  b. Average case
  c. Worst case

- to denote time & space complexity, asymptotic notations are used:

Omega (Ω)	Big O (O)	Theta (Θ)
↓	↓	↓
lower bound	upper bound	Avg / Tight bound

Time complexity

- time is directly proportional to number of iterations of the loops used in an algorithm
- To find time complexity/requirement of the algorithm count number of iterations of the loops

1. Find factorial of given number

```
fact = 1
for( i = 1; i <= n ; i++)
        fact *= i;
sysout (fact)
```

No. of iterations = n

Time ∝ iterations

Time ∝ n

$T(n) = O(n)$ ← linear growth

2. Print 2D array on console

```
for( i=0 ; i< m ; i++)
    for(j=0; j < n ; j++)
        sysout(arr[i][j]);
```

No. of iterations for outer loop = m

No of iterations for inner loop = n

Total No. of iteration = m * n

Time ∝ m * n

$T(m,n) = O(m*n)$

if $m \approx n$

Total iteration = $n * n = n^2$

Time ∝ n^2

$T(n) = O(n^2)$ ← quadratic growth

Time complexity

3. Add two numbers

```
int add (int n1 , int n2)
    return  n1 + n2;
}
```

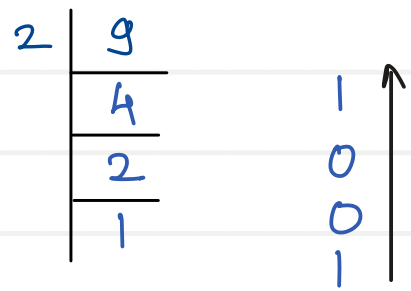
- need same amount of time irrespective of input values.
- constant time requirement which is denoted by O(1)

4. Print table of given number

```
void printTable( int n ) {
    for( i=1 ; i<=10 ; i++)
        sysout( i * n );
}
```

- loop will iterate fixed/ constant number of times.
- constant time requirement which will be denoted by O(1)

5. Print binary of decimal number



$(9)_{10} = (1001)_2$

```
while( n > 0 ) {
    rem = n % 2;
    n /= 2;
}
```

n	n>0	rem
9	T	1
4	T	0
2	T	0
1	T	1
0	F	

$n = n, n/2, n/4, n/8 \ldots\ldots$

$= n/2^0, n/2^1, n/2^2 \ldots\ldots n/2^{itr}$

$1 = n/2^{itr}$

$2^{itr} = n$

$\log 2^{itr} = \log n$

$itr \log 2 = \log n$

$itr = \frac{\log n}{\log 2}$

$Time \propto \frac{1}{\log 2} \log n$

$T(n) = O(\log n)$ ← logarithm growth

```
for(i=0; i<n ; i++)
for(i=n; i>0 ; i--)
```
} O(n)

```
for(i=0; i<n ; i+=20)
```
– O(n)

```
for( i=1 ; i<=10 ; i++)
```
– O(1)

```
for( i=n; i>0 ; i/=2)
for( i=1 ; i<n ; i*=2)
```
} O(log n)

```
for(i=1 ; i<n ; i++) → n
    for(j=1 ; j<n ; j++) → n
```
$= n^2$

$\downarrow$

$O(n^2)$

```
for(i=1 ; i<n ; i++); → n
for(j=1 ; j<n ; j++); → n
```
$= 2n$

$T \propto 2n \rightarrow O(n)$

```
for( i=0 ; i<n ; i++) → n
    for( j=n ; j>0 ; j/=2) → log n
```

itr = $n \log n \rightarrow O(n \log n)$

```
for(i=0; i<n ; i++)
    for(j=i ; j<n ; j++)
```

i	j	itr of inner loop
0	0 → n-1	n
1	1 → n-1	n-1
2	2 → n-1	n-2
⋮		⋮
n-1	n-1 → n-1	1

Total itrs = $1+2+3+\cdots+n$

$$= \frac{n(n+1)}{2}$$

$$= \frac{n^2+n}{2}$$

$$\text{Time} \propto \frac{1}{2}(n^2+n)$$

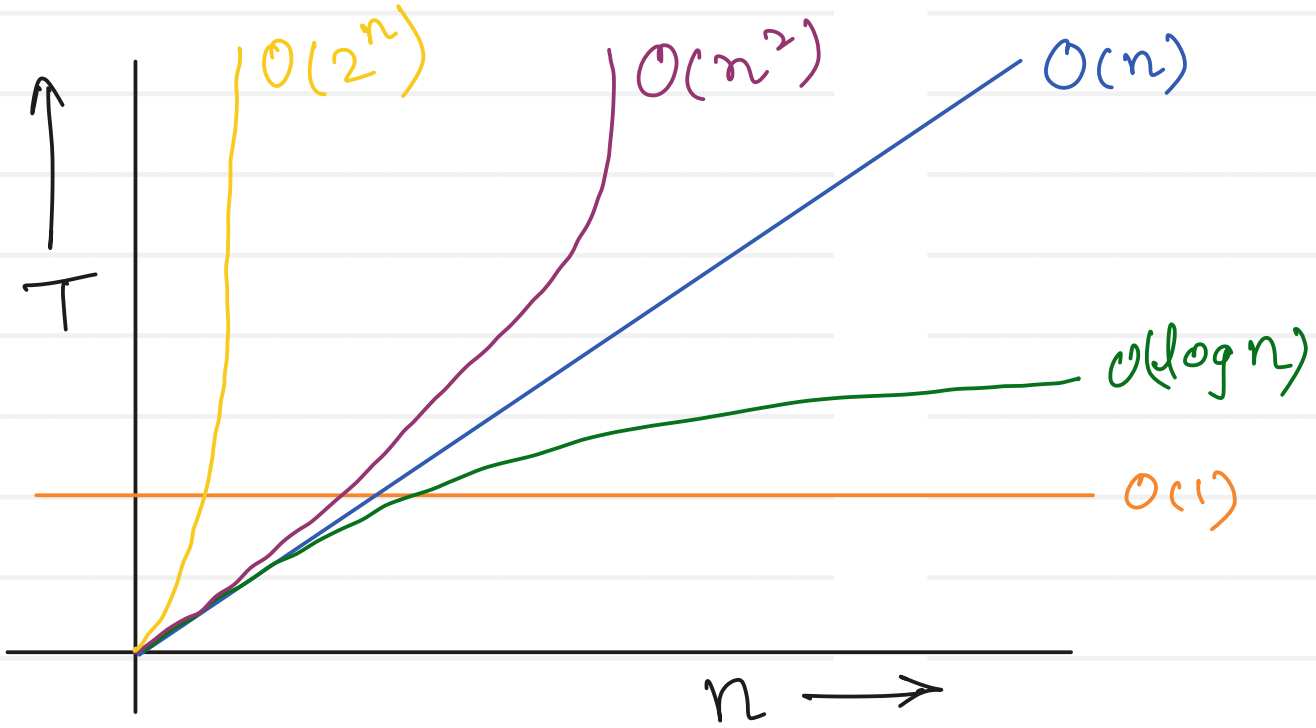
$$T(n) = O(n^2)$$

- while writing time & space complexity in a mathematical polynomial consider only highest power(degree) term. because it highest growing term in the polynomial

n	n^2
1	1
10	100
100	10000
1000	1000000

Time complexity

Time complexities :
O(1), O(log n), O(n), O(n log n), O(n^2), O(n^3), …., O(2^n), ……



Modification : + or - : time complexity is in terms of n
Modification : * or / : time complexity is in terms of log n

Space complexity

- Finding approximate space requirement of the algorithm to execute inside memory

Total space = Input space + Auxiliary space

Input space → space required to store input (data)

Auxiliary space → extra space required to process the input

```
int arraySum( int arr[], int N ) {
    int sum = 0;
    for( i=0; i<N; i++ )
        sum += arr[i];
    return sum;
}
```

input variable = arr

processing variables = N, sum, i

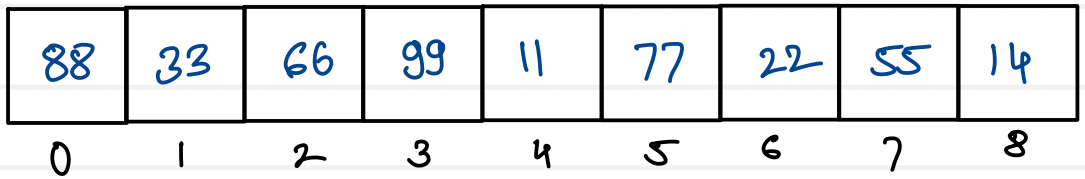
Auxilliary space = 3 units → constant

$S(n) = O(1)$

Linear search

Algorithm :

1. decide/ take key from user
2. traverse collection from one end to another
3. compare key with each element of collection.
   - 3.1 if key is matching
     return true/index
   - 3.2 if key is not matching
     go to next element
4. if collection is finished, key is not found
   return false/-1



key == arr[i]

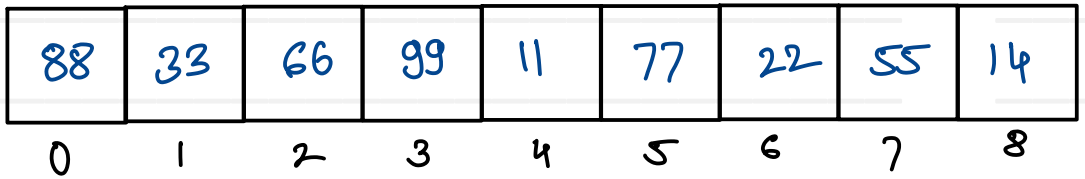
11 (Key)

i = 0, 1, 2, 3, 4 ↑ key is found

100 (Key)

i = 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 ↑ key is not found

Linear search - Analysis




```
for(i=0; i<n; i++) {
```

No. of iterations = n

Time $\propto$ n

$T(n) = O(n)$

processing variables = key, i, N

Auxilliary space is constant

$S(n) = O(1)$

key = 88 → best case → itrs = 1 → O(1)

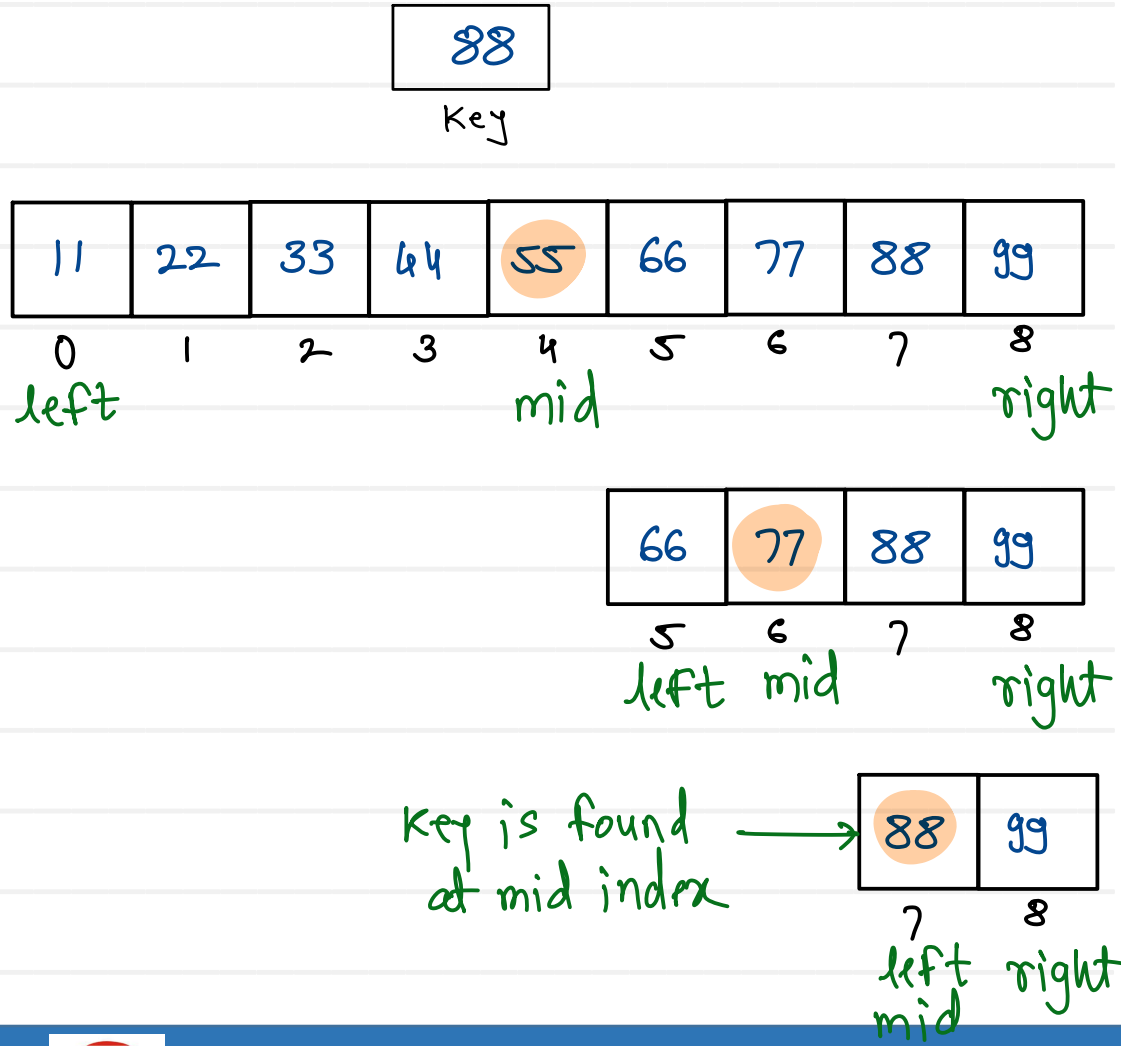
key = 11 → Avg case → itrs = n/2 → O(n)

key = 14, key = 100 → worst case → itrs = n → O(n)

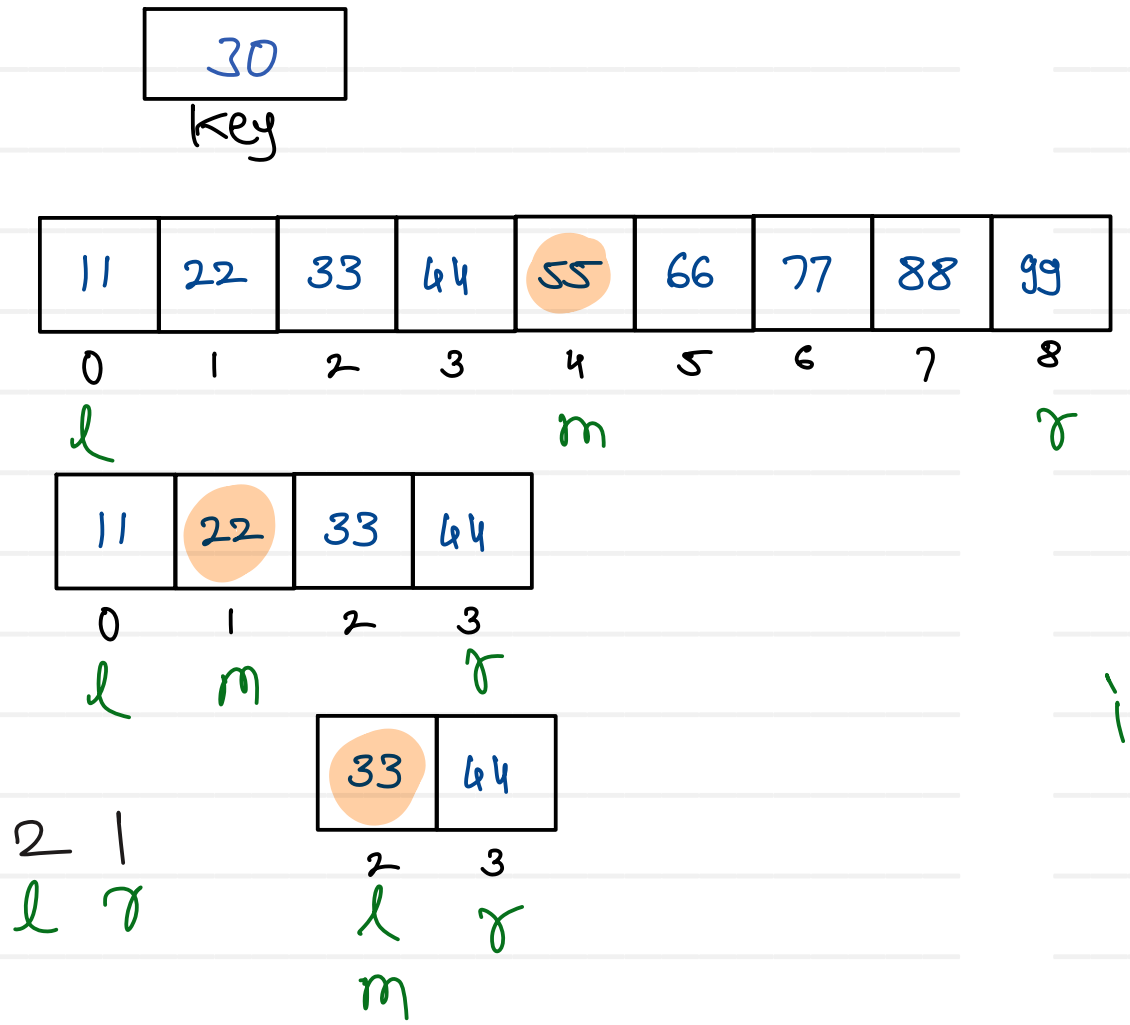
Binary search

1. take key from user
2. divide array into two parts
   (find middle element)
3. compare middle element with key
3.1 if key is matching
   return index (mid)
3.2 if key is less than middle element
   search key in left partition
3.3 if key is greater than middle element
   search key in right partition
3.4 if key is not matching
   return -1

mid = (left+right) / 2



Binary search



Array : left → right

left partition : l → m-1

right partition : m+1 → r

valid partition : left <= right

invalid partition : left > right

Binary search

```
l=0 , r=8 , m;
while(l <= r) {
    m = (l+r) /2;
    if( key == arr[m]
        return m;
    else if( key < arr[m])
        right= m-1;
    else
        left = m+1;
}
return -1;
```

	11	22	33	44	55	66	77	88	99
	0	1	2	3	4	5	6	7	8

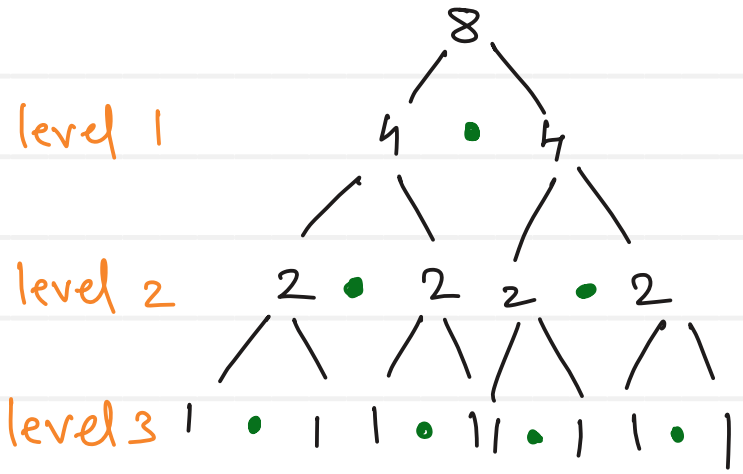
key = 88

l	r	l <= r	m
0	8	T	4
5	8	T	6
7	8	T	7

key = 30

l	r	l <= r	m
0	8	T	4
0	3	T	1
2	3	T	2
2	1	F	

Binary search - Analysis



No. of comps/itrs = No. of levels of division

$$2^l = n$$

l – no. of levels
n – no. of elements

$$2^l = n$$

$$\log 2^l = \log n$$

$$l \log 2 = \log n$$

$$l = \frac{\log n}{\log 2}$$

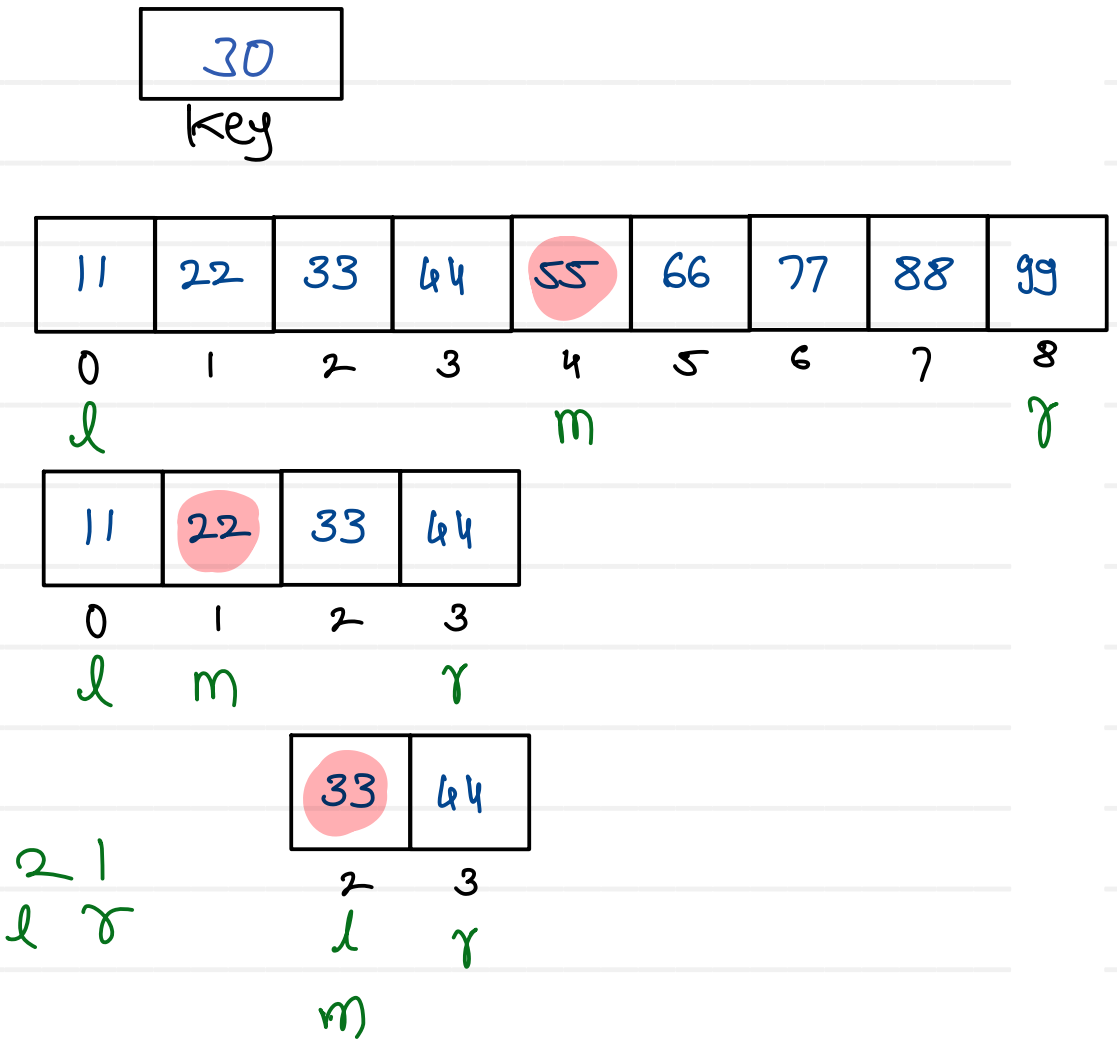
Time $\propto$ comps

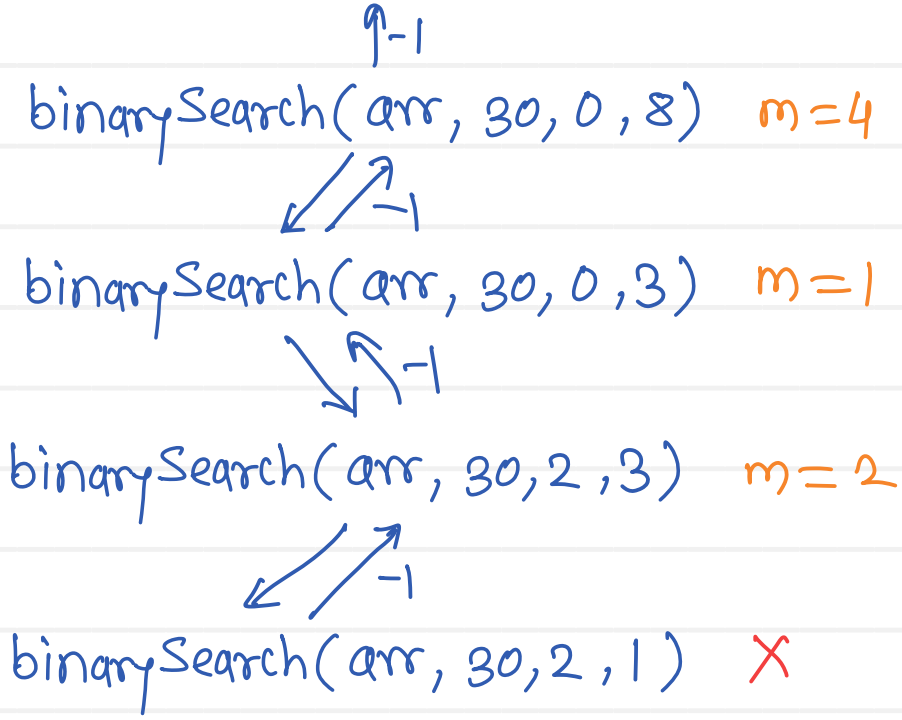
Time $\propto \frac{\log n}{\log 2}$

$$T(n) = O(\log n)$$ – Avg, worst

$O(1)$ – best

Recursive binary search





Searching algorithms analysis

- Time is directly proportional to number of comparisons
- For searching and sorting algorithms, count number of comparisons done

1. Linear search
   - Best case - if key is found at few initial locations → O(1)
   - Average case - if key is found at middle locations → O(n)
   - Worst case - if key is found at last few locations / key is not found → O(n)

S(n)=O(1)

2. Binary search
   - Best case - if key is found at first few levels → O(1)
   - Average case - if key is found at middle levels → O(log n)
   - Worst case - if key is found at last level / not found → O(log n)

S(n)=O(1)

- function calling itself
- recursion can be used when we know
  1. formula / process in terms of itself
  2. terminating / base condition
- for every recursive call, one stack frame / FAR will be created on a process stack.

$$n! = n * (n-1)!$$

$$0! = 1! = 1$$

$$b^i = b * b^{i-1}$$

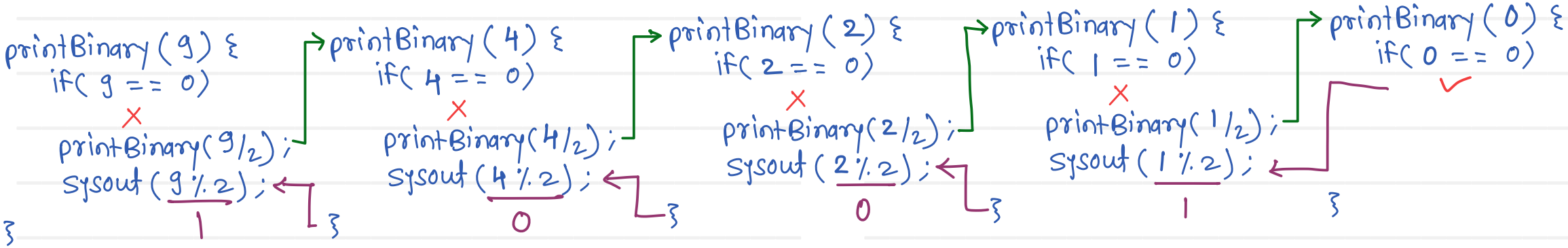
$$b^0 = 1 \quad , \quad b^1 = b$$

$$T_n = T_{n-1} + T_{n-2}$$

$$T_1 = T_2 = 1$$

Recursion - print binary

```
void printBinary(int n) {
    if(n == 0)
        return;
    printBinary(n/2);
    sysout(n % 2);
}
```



Recursion analysis

Iterative approach

- loops are used to implement algorithms

Time $\propto$ No. of iterations of the loop

```
int factorial ( int n) {
    int fact = 1
    for( i=1; i<=n; i++)
        fact *= i;
    return fact;
}
```

No. of iterations = n

Time $\propto$ n

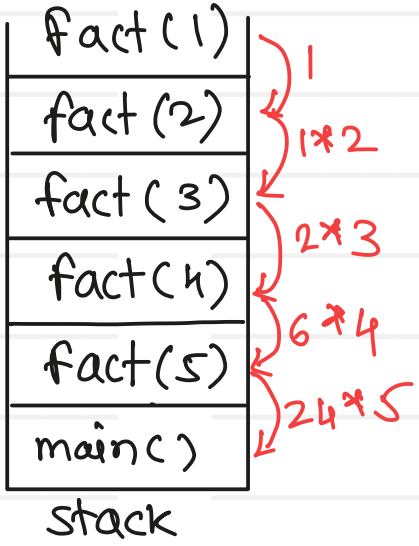
$T(n) = O(n)$

Extra space = i, fact

$S(n) = O(1)$

Recursive approach

- recursion is used to implement algorithms



Time $\propto$ No. of recursive calls of a function

```
int factorial ( int n) {
    if( n == 1)
        return 1;
    return n * factorial (n-1);
}
```

No. of recursive calls = n

Time $\propto$ n

$T(n) = O(n)$

n-FARs will be created on stack

$S(n) = O(n)$

Types of recursion

Recursion

- Direct Recursion
  (same function is called within function)

```
e.g. fun() {
       fun()
     }
```

  - Tail Recursion
    if recursive call is last statement in recursive function
  - Non tail (head) Recursion
    if recursive call is not last statement in recursive function

- Indirect Recursion
  (more than one function calls each other in circular way)

```
e.g. fun1() {
       fun2();
     }

     fun2() {
       fun1();
     }
```

  - it is used very rarely

Applications :
1. even odd processing
2. state machine
3. big/multi player games
4. Complex expressin parsing

Tail Recursion

```
void printNumber(int n)
    if(n > 100)
        return
    sysout(n);
    printNumber(n+1);
}
```

Non Tail (head) Recursion

```
void printNumber(int n)
    if(n > 100)
        return
    printNumber(n+1);
    sysout(n);
}
```

- tail recursion will be more efficient than non tail recursion.
- compiler will TCO (Tail Call Optimization) for tail recursions